

Mobile KI-Bildverarbeitung zur Qualitätskontrolle in der Produktion

Objekterkennung mit KI

- Computer erkennt Objekte oder Fehler automatisch in Bildern
- Nutzung von künstlicher Intelligenz und trainierten Modellen
- Analyse von Kamerabildern möglich
- Einsatz zur automatischen Qualitätskontrolle in der Produktion

Was ist YOLO?

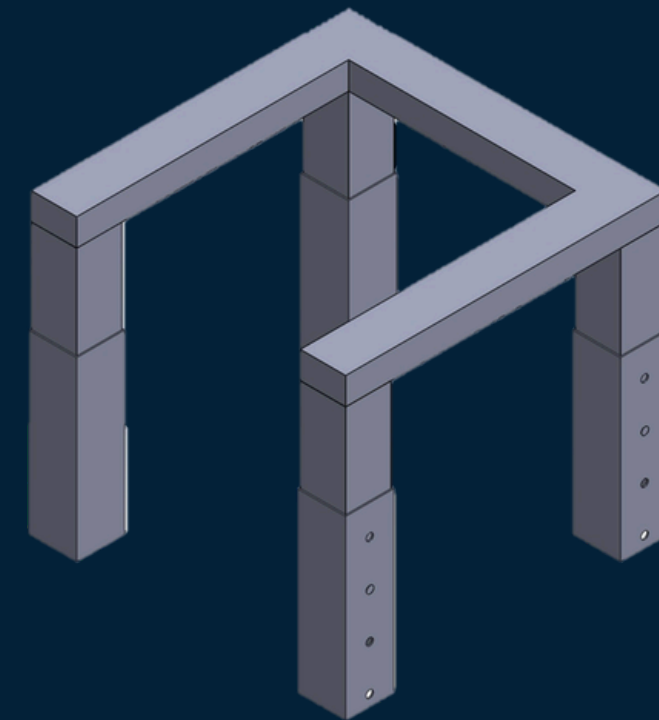


- YOLO = „You Only Look Once“
- KI-Modell zur schnellen Objekterkennung in Bildern
- Analysiert ein Bild in nur einem Durchgang

Vorteile der automatisierten Kontrolle

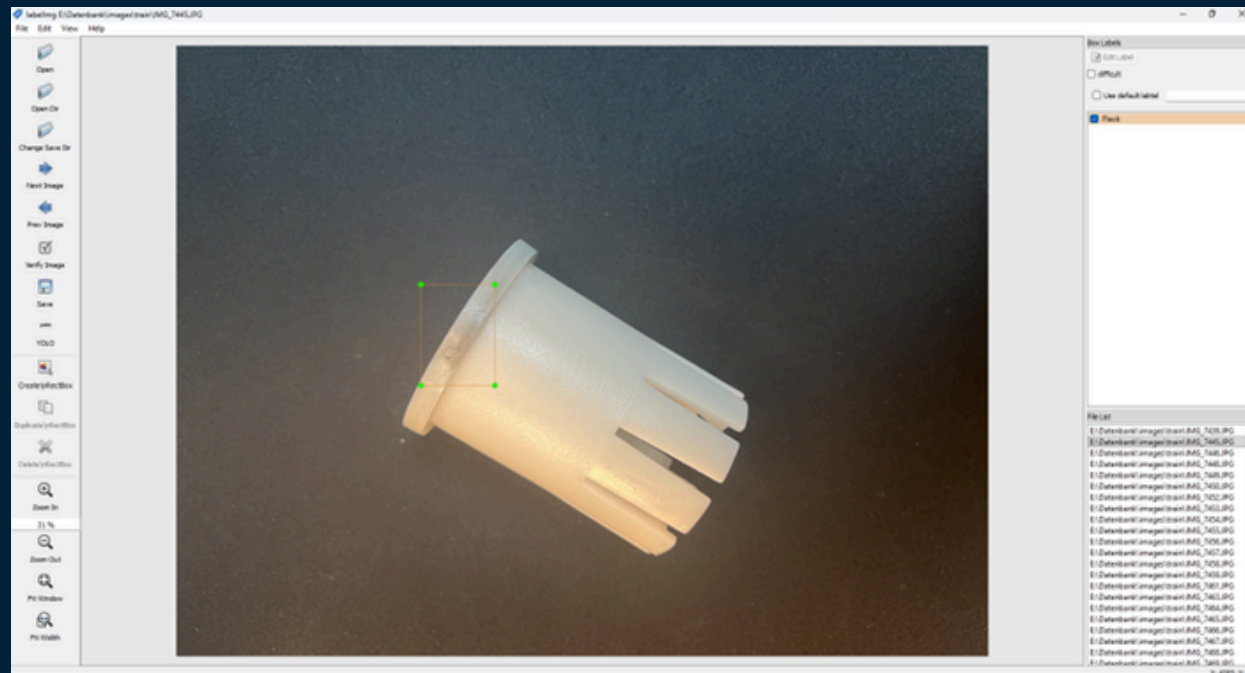
- Schnelle Auswertung großer Stückzahlen
- Objektive Fehlerbewertung
- Weniger Ausschuss in der Produktion
- Flexible Nutzung für unterschiedliche Produkte

Hardware



- Schnittstelle der Kameras/Beleuchtung und Software
- Höhenverstellbar
- Mobil einsetzbar direkt an Produktionslinien
- Geringer Platzbedarf, niedriger Energieverbrauch

Training der KI



- Bilder von fehlerhaften und fehlerfreien Produkten sammeln
- KI-Modell mit Beispielen trainieren
- 70% Bilder für Training, 30% Bilder für Überprüfung
- Regelmäßiges Nachtrainieren verbessert Genauigkeit

Software

```
main.py > ...
1 #Import aller benötigter Sachen
2 import cv2
3 import numpy as np
4 from ultralytics import YOLO
5
6 model = YOLO(r"E:\Seminararbeit\runs\detect\train20\weights\best.pt") #Pfad zur KI
7
8 model.model.names = {0: 'Kratzer', 1: 'fehlendes Material', 2: 'Fleck', 3: 'Anguss noch dran'} #Beschreibung der Klassen
9
10 cap1 = cv2.VideoCapture(0) #Initialisierung der Kameras
11 cap2 = cv2.VideoCapture(1)
12
13 while True:
14     ret1, img1 = cap1.read()
15     ret2, img2 = cap2.read()
16
17     if not ret1 or not ret2:
18         break
19
20     TARGET_HEIGHT = 480
21
22     img1 = cv2.resize(img1, (int(img1.shape[1] * TARGET_HEIGHT / img1.shape[0]), TARGET_HEIGHT))
23     img2 = cv2.resize(img2, (int(img2.shape[1] * TARGET_HEIGHT / img2.shape[0]), TARGET_HEIGHT))
24
25     # YOLO Inferenz
26     results1 = model(img1, conf=0.25)
27     results2 = model(img2, conf=0.25)
28
29     # Bounding Boxes einzeichnen
30     annotated1 = results1[0].plot()
31     annotated2 = results2[0].plot()
32
33     # Zusammenfügen
34     combined = np.hstack((annotated1, annotated2))
35
36     cv2.imshow("Beide Webcams", combined)
37
38     if cv2.waitKey(1) & 0xFF == ord("q"):
39         break
40
41 cap1.release()
42 cap2.release()
43 cv2.destroyAllWindows()
44
45 #Gibt die Kameras nach Beenden des Programms wieder frei
46 cap1.release()
47 cap2.release()
48 cv2.destroyAllWindows()
```

Probleme

- Viele Trainingsbilder für gute Ergebnisse notwendig
- Lichtverhältnisse können die Erkennung beeinflussen
- Unterschiedliche Perspektiven erschweren die Analyse
- Fehler können manchmal falsch erkannt werden (Fehlklassifikation)

- Programmiersprache Python zur Entwicklung des Systems
- Bibliotheken für Bildverarbeitung und KI
- Training eines Modells zur Fehlererkennung
- Ausgabe eines Ergebnisses („Kratzer“, „Anguss noch dran“, „fehlendes Material“, „Farbe“)